



UNIVERSIDAD NACIONAL DE CÓRDOBA

FACULTAD DE CIENCIAS EXACTAS, FÍSICAS Y NATURALES

Sistemas de Computación

Trabajo Práctico 5

Device Drivers

Integrantes

Gonzalez Leahy María Clara

Nuñez Anna

Viguzzi Ignacio Joaquin

INTRODUCCIÓN

En este trabajo práctico se desarrollará un Driver de Dispositivo de Caracteres (Character Device Driver, CDD) para Linux, capaz de adquirir periódicamente dos señales externas con un período de muestreo de un segundo. A través de una aplicación de usuario, será posible seleccionar cuál de las señales se desea monitorear, leer los datos proporcionados por el driver y representarlos gráficamente en función del tiempo.

El desarrollo se realizará siguiendo una metodología de compilación cruzada, utilizando una computadora anfitriona para la programación y compilación del software, y una Raspberry Pi como plataforma objetivo para la ejecución y prueba del sistema. De esta manera, se busca aplicar conceptos de programación de controladores de dispositivos, interacción entre espacio de usuario y espacio de kernel, y adquisición de datos en sistemas embebidos basados en Linux.

DESARROLLO

Para este trabajo se utilizó una placa Raspberry Pi 5 con el sistema operativo Raspberry Pi OS Lite. El objetivo es crear y cargar correctamente un Character Device Driver en el kernel que envíe 2 señales digitales por pines GPIO y las lea como entradas por otros 2 pines. Además, se dispone de una aplicación web para elegir la señal a analizar y graficar en función del tiempo.

Ya que el sistema en el que se desarrolló el código cuenta con un procesador x86_64 y la placa tiene un procesador Broadcom BCM2712 de arquitectura ARM, se debieron implementar mecanismos de compilación cruzada para el desarrollo del módulo de kernel. Para esto se debió utilizar el compilador **gcc-aarch64-linux-gnu** y un Makefile indicando el compilador, la arquitectura objetivo y la ubicación de los headers del kernel de la placa que debieron ser importados al equipo host.

Device Tree Overlay

Ya que se trata de un driver que se cargará en el kernel del sistema, no se pueden utilizar librerías comunes para el manejo de los pines GPIO. En versiones anteriores del kernel esto se podía lograr fácilmente utilizando los identificadores de los pines y la API GPIO Legacy. Sin embargo, la API Legacy ha quedado obsoleta en nuevas versiones del kernel y, además, la Raspberry Pi 5 dejó de controlar los pines desde el procesador central e implementó un chip controlador dedicado al GPIO llamado RP1.

La forma moderna de poder manejar los pines desde el código es mediante la creación y compilado de un **Device Tree Overlay**. Se trata de un archivo que le indica al kernel el hardware presente, donde se encuentra en la memoria y que interrupciones o pines GPIO usa.

De esta forma, se crea un archivo `signal-sensors.dts` indicando los pines a usar y asignándoles nombres para facilitar su acceso, pasa por un proceso de precompilado para resolver los includes con:

```
cpp -p signal-sensors.dts -o signal-sensors_pre.dts
```

y finalmente se compila utilizando el compilador **Device-tree-compiler** con el comando:

Bash:

```
dtc -@ -I dts -O dtb -o signal-sensors.dtbo signal-sensors_pre.dts
```

Obteniendo un archivo dtbo que puede ser cargado en el sistema con el comando

```
sudo dtoverlay signal-sensors.dtbo.
```

Ya que la placa cuenta con solo 1gb de memoria RAM, en lugar de utilizar un sistema operativo con entorno de escritorio, se controló la placa desde el equipo host mediante ssh para la carga de los módulos.

A terminal window titled 'user@raspberrypi: ~/proyectos' with standard window controls. The terminal shows a user logging in via SSH to a Raspberry Pi. The system is Linux raspberrypi 6.18.33+rpt-rpi-2712 #1 SMP PREEMPT Debian 1:6.18.33-1+rpt1 (2026-06-01) aarch64. It displays the Debian GNU/Linux license and a web console URL. The user then navigates to the 'proyectos' directory and runs 'sudo dtoverlay signal-sensors.dtbo', providing a password when prompted. The prompt returns to the user's shell.

```
user@raspberrypi: ~/proyectos
user@Device01:~$ ssh user@raspberrypi.local
Linux raspberrypi 6.18.33+rpt-rpi-2712 #1 SMP PREEMPT Debian 1:6.18.33-1+rpt1 (2026-06-01) aarch64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Web console: https://raspberrypi:9090/ or https://192.168.1.17:9090/

Last login: Wed Jun 10 18:42:06 2026 from 192.168.1.8
user@raspberrypi:~ $ cd proyectos/
user@raspberrypi:~/proyectos $ sudo dtoverlay signal-sensors.dtbo
[sudo] password for user:
user@raspberrypi:~/proyectos $
```

Generado y sensado de señales

Una vez que se tienen definidos los pines que producirán y leerán las señales, se pueden integrar al código del driver. El mismo consta de un timer que aumenta un contador cada 10ms generando 2 señales cuadradas con periodo de 1s pero con diferente duty cycle, siendo de 50% en el pin 23 y 10% en el pin 24.

Una vez terminado se compila con el Makefile y se envía a la placa por ssh:

```
user@user-Latitude-5510: ~/programacion/raspberrypi
user@user-Latitude-5510:~/programacion/raspberrypi$ make
make -C /home/user/programacion/raspberrypi/rpi-kernel M=/home/user/programacion/raspberrypi ARCH=arm64 CROSS_COMPILE=aarch64-linux-gnu- modules
make[1]: Entering directory '/home/user/programacion/raspberrypi/rpi-kernel'
make[2]: Entering directory '/home/user/programacion/raspberrypi'
  CC [M]  signal-sensors.o
  MODPOST Module.symvers
  CC [M]  signal-sensors.mod.o
  CC [M]  .module-common.o
  LD [M]  signal-sensors.ko
make[2]: Leaving directory '/home/user/programacion/raspberrypi'
make[1]: Leaving directory '/home/user/programacion/raspberrypi/rpi-kernel'
user@user-Latitude-5510:~/programacion/raspberrypi$ scp signal-sensors.ko user@raspberry.local:/home/user/proyectos/
signal-sensors.ko                               100% 15KB 1.3MB/s 00:00
user@user-Latitude-5510:~/programacion/raspberrypi$
```

donde será finalmente cargado:

```
user@raspberry: ~/proyectos
user@raspberry:~/proyectos $ sudo dtoverlay signal-sensors.dtbo
user@raspberry:~/proyectos $ sudo insmod signal-sensors.ko
user@raspberry:~/proyectos $ dmesg | tail -2
[ 2444.669316] CDD: Device Tree entry matched! Initializing GPIOs...
[ 2444.669483] CDD: Registered correctly in /dev/signal_sensors
user@raspberry:~/proyectos $
```

El valor de la señal puede observado leyendo /dev/signal_sensors del sistema virtual de archivos de linux:

```
user@raspberrypi: ~/proyectos
user@raspberrypi:~/proyectos $ sudo cat /dev/signal_sensors
0
user@raspberrypi:~/proyectos $ sudo cat /dev/signal_sensors
1
user@raspberrypi:~/proyectos $
```

Además, se puede elegir la señal observada escribiendo al mismo archivo.

Aplicación web

Para la visualización de las señales, se creó un pequeño servidor http en python que correrá en la placa y muestra una página html que registra los valores de /dev/signal_sensors con suficiente frecuencia para no perder información, los grafica en función del tiempo y tiene botones para cambiar de señal:

```
user@raspberrypi: ~/proyectos/app
user@raspberrypi:~/proyectos/app $ python3 server.py
Server running on http://localhost:5000

```

Monitoreo de Señales - RPi5

Señal 1 Señal 2

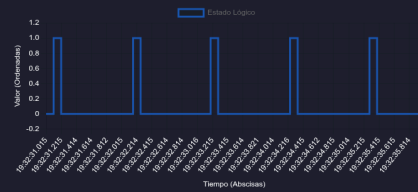
Visualizando: Señal 1



Monitoreo de Señales - RPi5

Señal 1 Señal 2

Visualizando: Señal 2



CONCLUSIÓN

En este trabajo práctico se logró desarrollar e implementar un Character Device Driver para Linux capaz de generar y adquirir dos señales digitales a través de los pines GPIO de una Raspberry Pi 5. Para ello fue necesario emplear técnicas de compilación cruzada, la creación de un Device Tree Overlay y la interacción entre espacio de usuario y espacio de kernel mediante un dispositivo de caracteres.

Además, se desarrolló una aplicación web que permite seleccionar la señal a monitorear y visualizar su evolución temporal de forma gráfica. De esta manera, se integraron conceptos de sistemas embebidos, programación de drivers en Linux, manejo de GPIO y comunicación entre aplicaciones de usuario y módulos del kernel, cumpliendo con los objetivos propuestos para el trabajo.